Universidade Franciscana
Somos universidade. Somos você.

SISTEMAS DA INFORMAÇÃO

COMPUTAÇÃO MÓVEL

Aula 05

Projeto CRUD - Atualizar/Excluir - Aluno

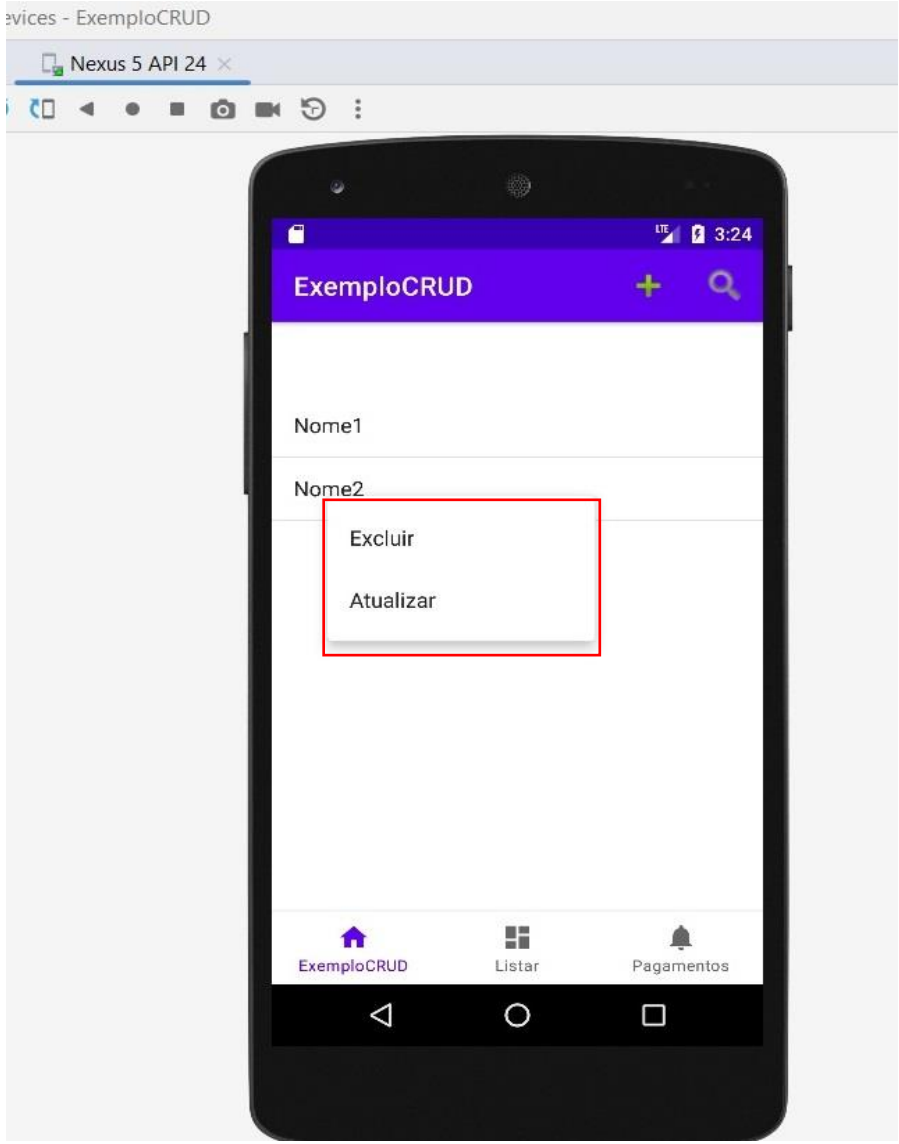
Professor: André Flores dos Santos.

CC BY NC SA

Santa Maria – RS 2026

Atualizar/Excluir

Vamos criar um menu, conforme o usuário clica/segura algum item da lista de alunos no ListView do (ListarAlunos.java) irá abrir uma opção de submenu para escolher entre ‘excluir’ ou ‘atualizar’

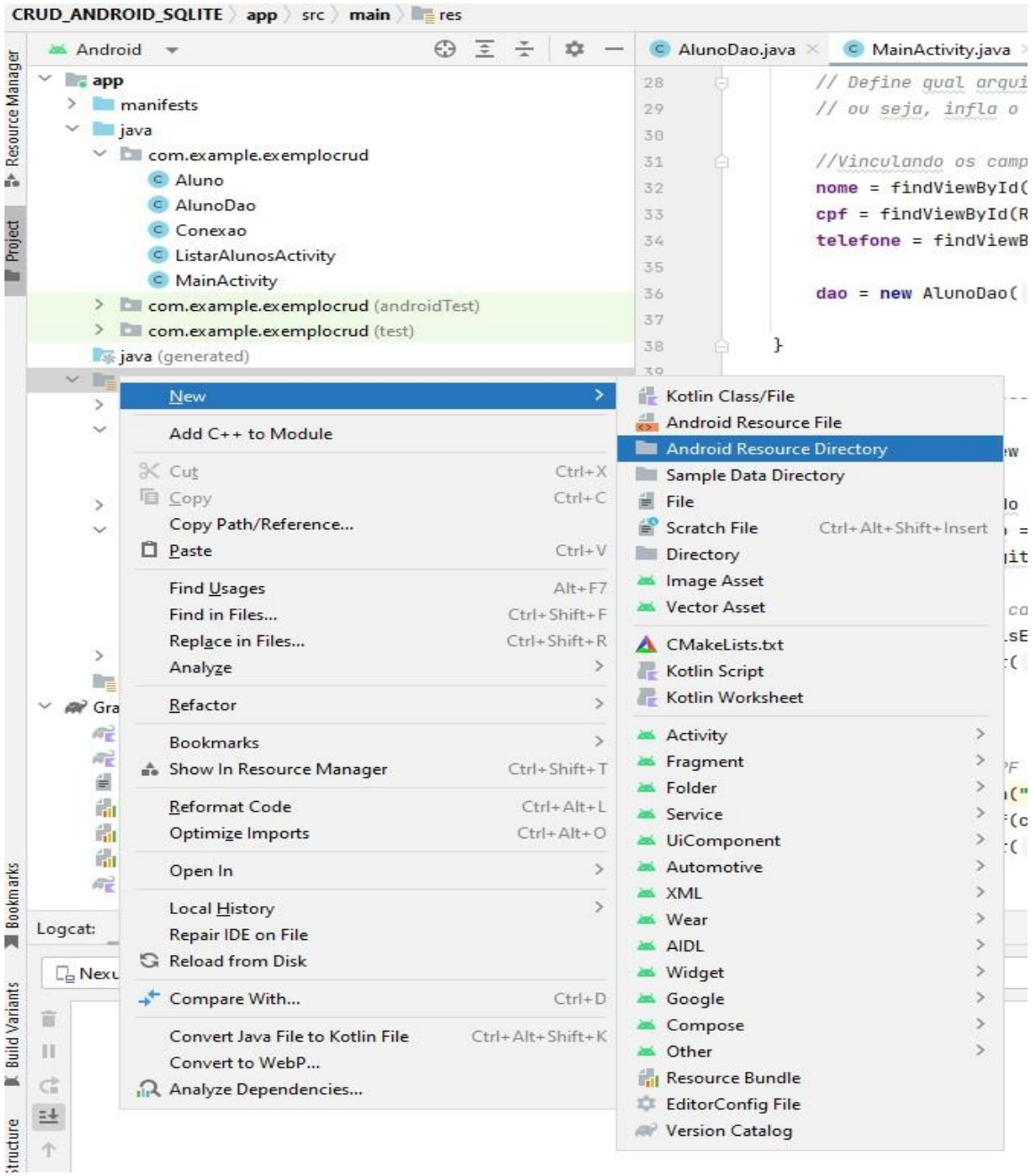


Atualizar/Excluir

Passo 01 – Criar um menu chamado ‘menu_contexto.xml’ na pasta ‘app>res>menu’.

Se a pasta app>res>menu não existir, vamos cria-la.

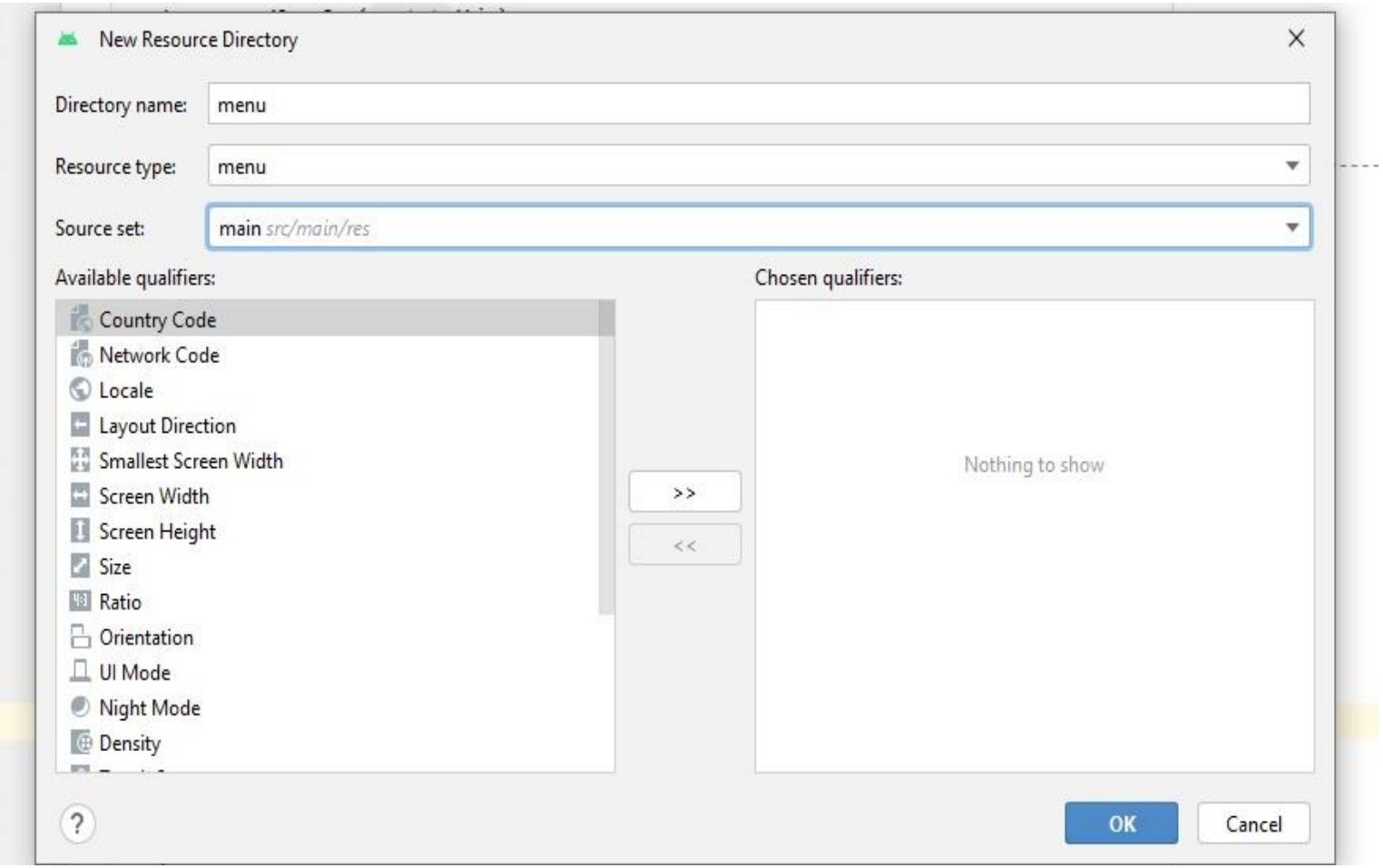
Clicar com botão direito no pasta app>res e escolher ‘new>Android Resource Directory
Name> menu
Type: menu



Atualizar/Excluir

Passo 02 –

Name: ‘menu’ e type: ‘menu’

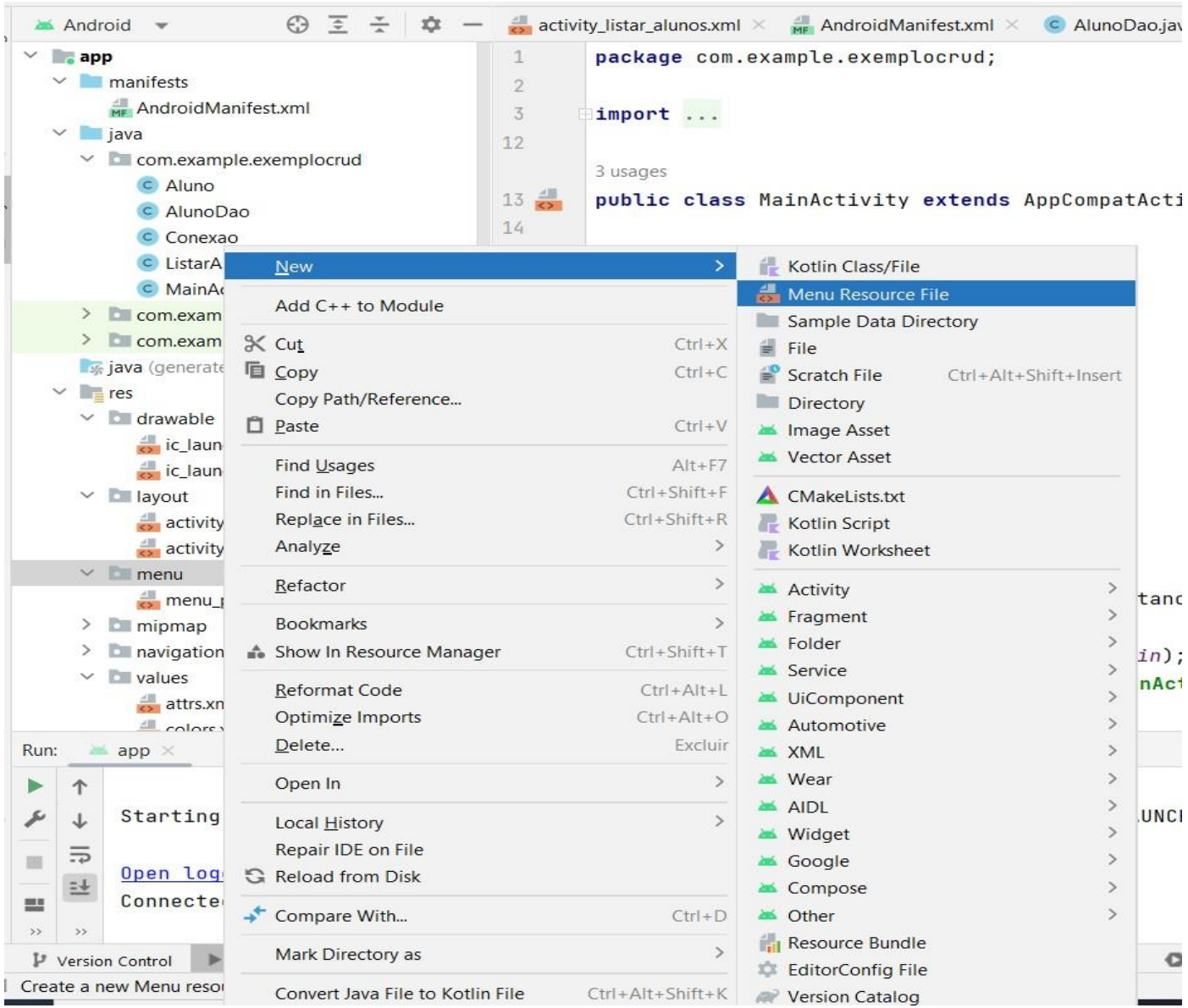


Atualizar/Excluir

Passo 03 – Criar um menu chamado ‘menu_contexto.xml’ na pasta ‘app>res>menu’.

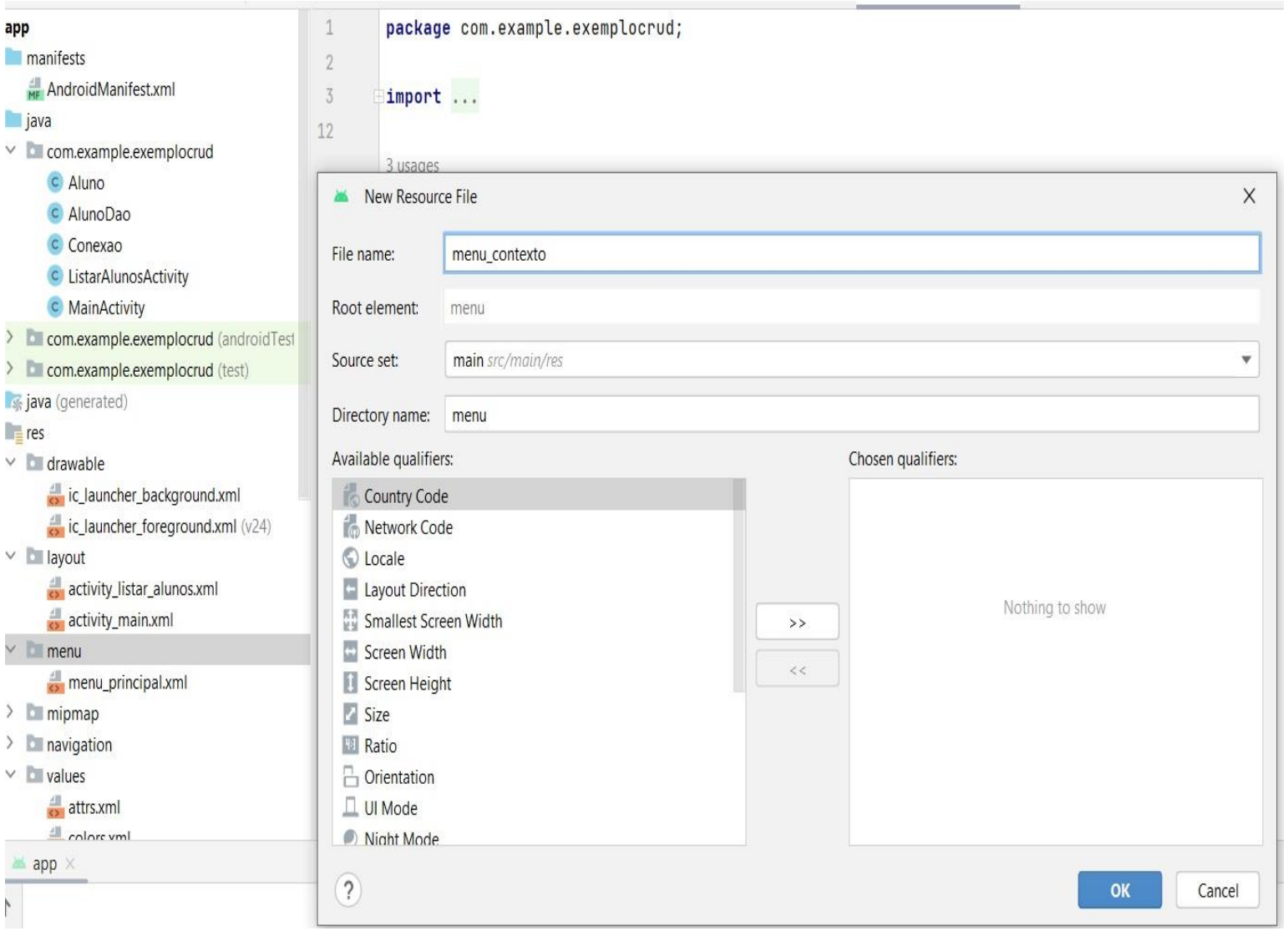
Agora clicar com o botão direito na pasta app>res>menu

New>Menu Resource File



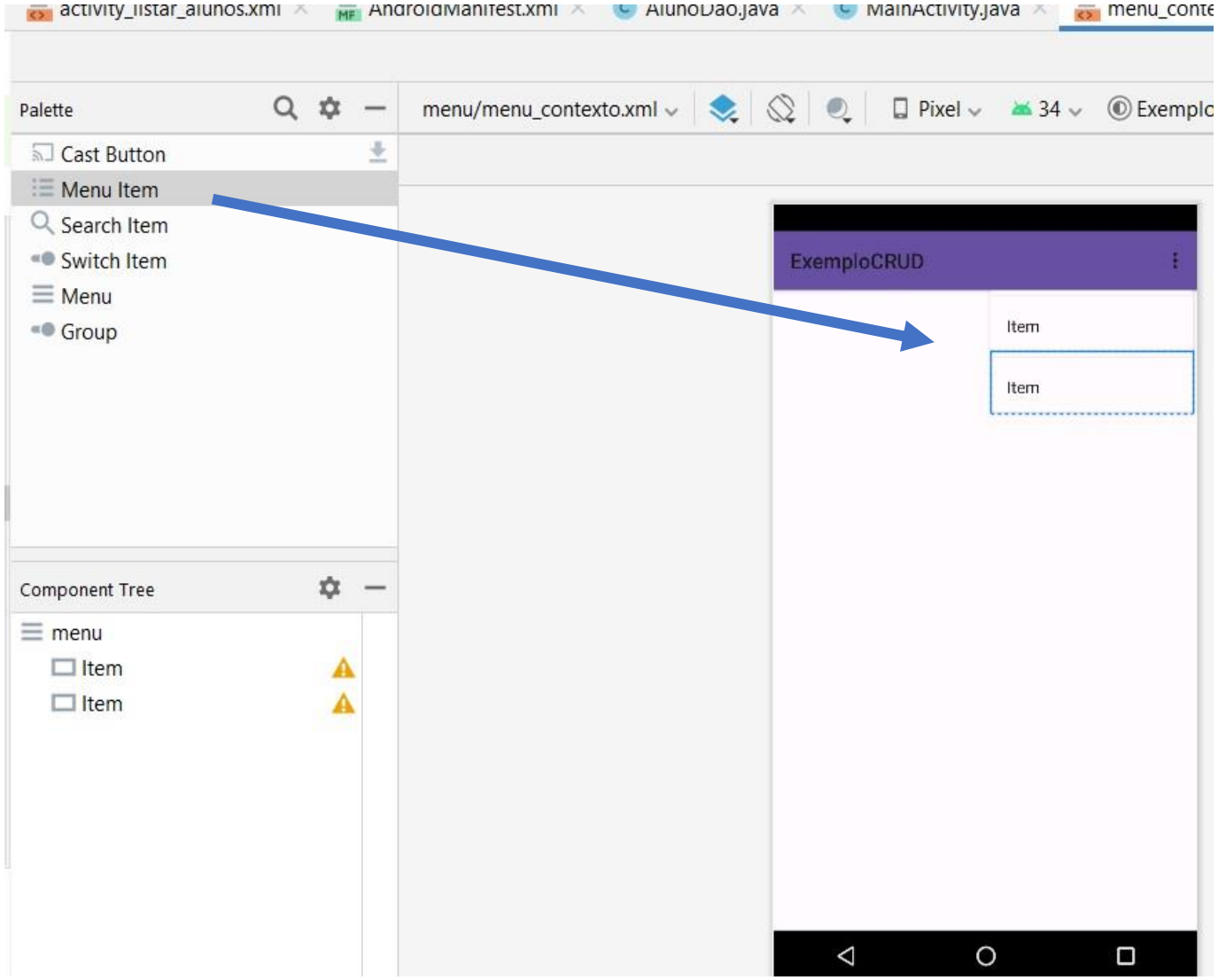
Atualizar/Excluir

Passo 04 – Preencher o nome‘menu_contexto.xml’ na pasta ‘app>res>menu’.



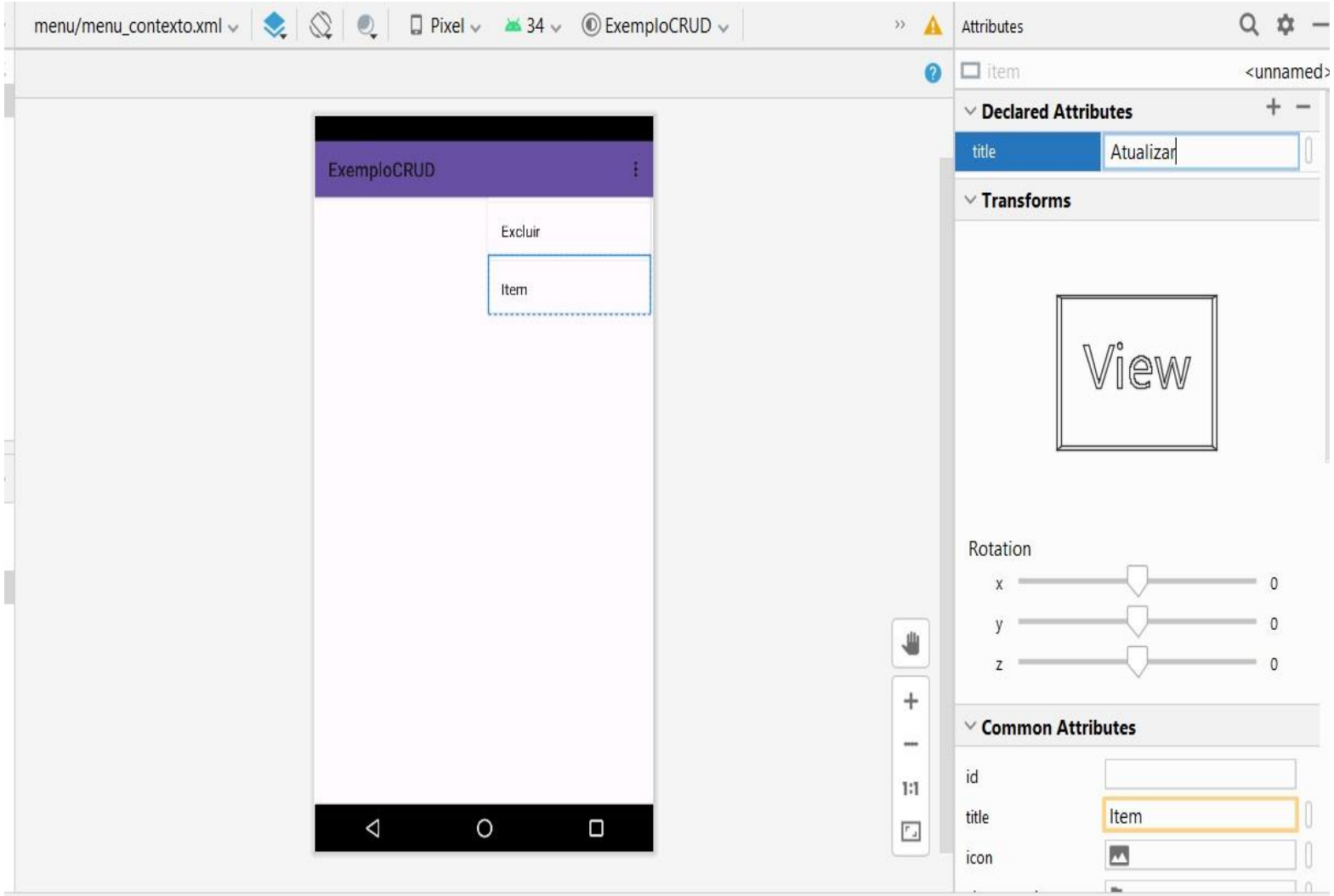
Atualizar/Excluir

Passo 05 – arrastar ‘2’ ‘menu item’ para dentro do menu após ele ser selecionado.



Atualizar/Excluir

Passo 06 – Preencher os nomes do campo title ‘Excluir’ e ‘Atualizar’ de cada menu item.



Excluir

Próximo passo – Criar o método para inflar o menu quando o item é pressionado, **dentro da Classe 'ListarAlunosActivity.java', após o método onCreate().**

```
//METODO MENU_CONTEXTO PARA INFLAR O MENU QUANDO ITEM PRESSIONADO
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo){
    // Chama o método da superclasse (neste caso, o método onCreateContextMenu da classe pai).
    // Isso é importante para garantir que qualquer comportamento padrão do método na superclasse
    // (por exemplo, qualquer configuração padrão de menu que a superclasse realiza) seja executado antes
    // de você adicionar suas próprias ações ao menu.
    super.onCreateContextMenu(menu, v, menuInfo);

    // Cria um objeto MenuInflater, que é responsável por inflar (converter um arquivo XML de menu em um objeto Menu)
    // o menu de contexto a partir de um arquivo XML de menu que você criou anteriormente.
    MenuInflater i = getMenuInflater();

    // O método inflate do MenuInflater é usado para inflar o menu de contexto.
    // Aqui, você está especificando o recurso XML (R.menu.menu_contexto) que define as opções de menu
    // que aparecerão quando um item da lista for pressionado.
    i.inflate(R.menu.menu_contexto, menu); //Aqui coloca o nome do menu que havia sido configurado
}
```

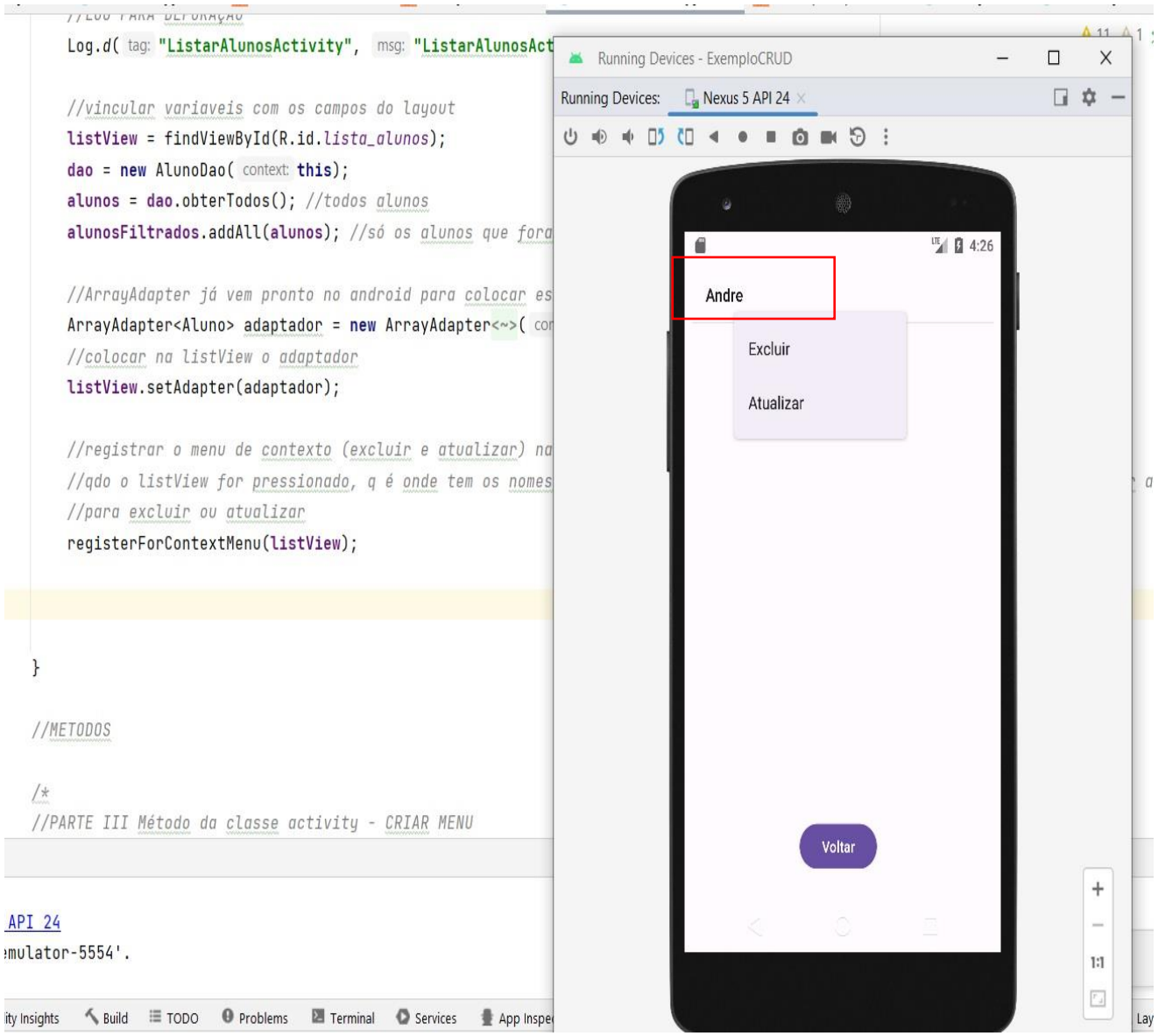
Excluir

Próximo passo – O menu sempre está ligado a uma ‘view’, então agora vamos configurar esta parte dentro do método ‘onCreate’ da Classe ListarAlunoActivity().

```
//colocar dentro do método onCreate() do ListarAlunosActivity
//registrar o menu de contexto (excluir e atualizar) na listview
registerForContextMenu(listView);
```

Excluir

Resultado – Ao ficar pressionando um item da lista de alunos o ‘Menu_contexto’ já abre com os itens que construímos. O próximo passo é configurar as ações de cada um que ainda não estão prontas.



Excluir

Declarar dentro da classe ListarAlunosActivity, uma lista de alunos filtrados, para podermos excluir o item selecionado e carregar novamente no listview atualizado.

Logo após dentro do onCreate() vincular os alunos retornados do método obterTodos() a essa lista.

```
public class ListarAlunosActivity extends AppCompatActivity {

    private ListView listView;
    private AlunoDao dao;
    private List<Aluno> alunos;
    private List<Aluno> alunosFiltrados = new ArrayList<>();
    // =new ArrayList<>() Para não iniciar como null

    //continua o código
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_listar_alunos);
    // Define qual arquivo de layout XML será usado para esta Activity,

    //vincular variaveis com os campos do layout
    listView = findViewById(R.id.lista_alunos); //lista_alunos é o id do listview
    dao = new AlunoDao(this);
    alunos = dao.obterTodos(); //todos alunos
    alunosFiltrados.addAll(alunos); //só os alunos que foram consultados

}
```

Excluir

Criar o método ‘excluir()’ dentro do ListarAlunosActivity

```
public void excluir(MenuItem item){
    // Obtém informações do item selecionado no menu de contexto.
    // O objeto `menuInfo` contém a posição do item na lista.
    AdapterView.AdapterContextMenuInfo menuInfo = (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();

    // Obtém o aluno que será excluído a partir da lista filtrada.
    final Aluno alunoExcluir = alunosFiltrados.get(menuInfo.position);

    // Exibe um alerta de confirmação antes de excluir o aluno
    AlertDialog dialog = new AlertDialog.Builder(this)
            .setTitle("Atenção") // Título do alerta
            .setMessage("Realmente deseja excluir o aluno?") // Mensagem de confirmação
            .setNegativeButton("NÃO",null) // Caso o usuário clique em "NÃO", fecha o alerta sem fazer nada.
            .setPositiveButton("SIM", new DialogInterface.OnClickListener() {
                @Override
                public void onClick(DialogInterface dialog, int which) {
                    // Remove o aluno da lista filtrada
                    alunosFiltrados.remove(alunoExcluir);
                    // Remove o aluno da lista principal
                    alunos.remove(alunoExcluir);
                    // Exclui o aluno do banco de dados
                    dao.excluir(alunoExcluir);
                    // Atualiza a ListView para refletir a exclusão
                    listView.invalidateViews();
                }
            } ).create(); // Cria a caixa de diálogo
    dialog.show(); // Exibe o alerta na tela
}
```

Excluir

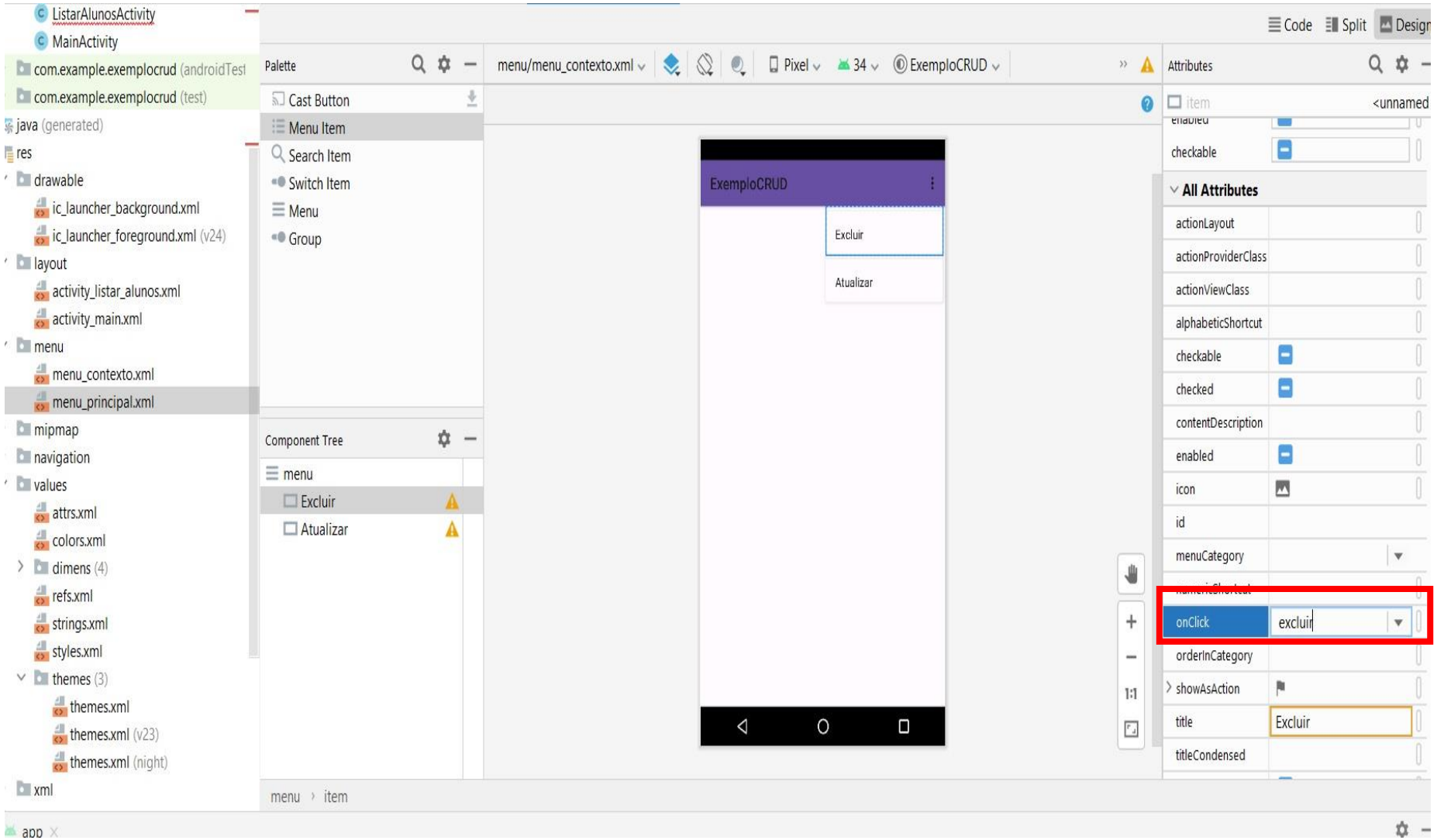
Criar o método Excluir dentro AlunoDao

E depois temos que configurar algumas coisas em outras classes
e o evento ‘OnClick’ do botão de menu ‘Excluir’

```
public void excluir(Aluno a){
   banco.delete("aluno", "id = ?",new String[]{a.getId().toString()}); // no lugar do ? vai colocar o id do aluno
}
```

Excluir

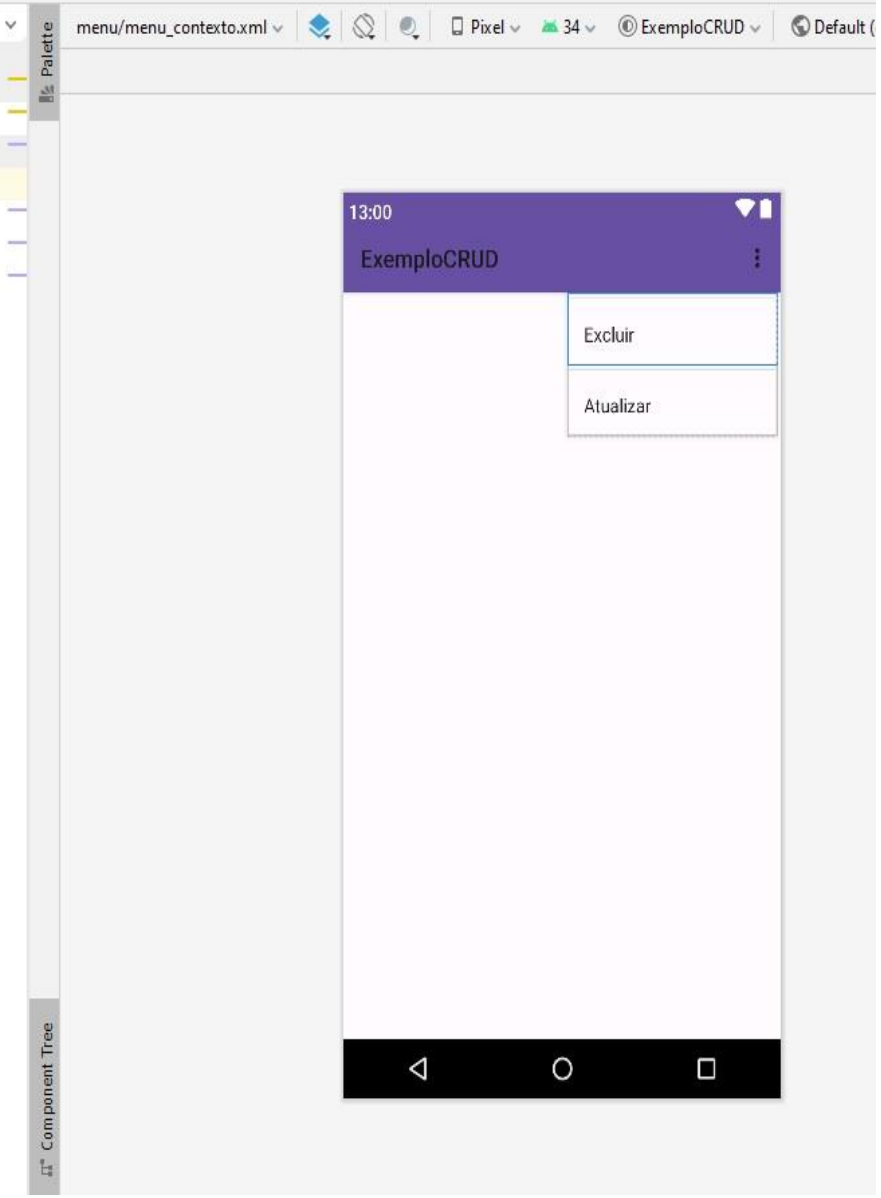
Configurar o evento ‘OnClick’ do botão de menu ‘Excluir’. Verificar se foi incluído também no código do Menu_Contexto.xml



Excluir

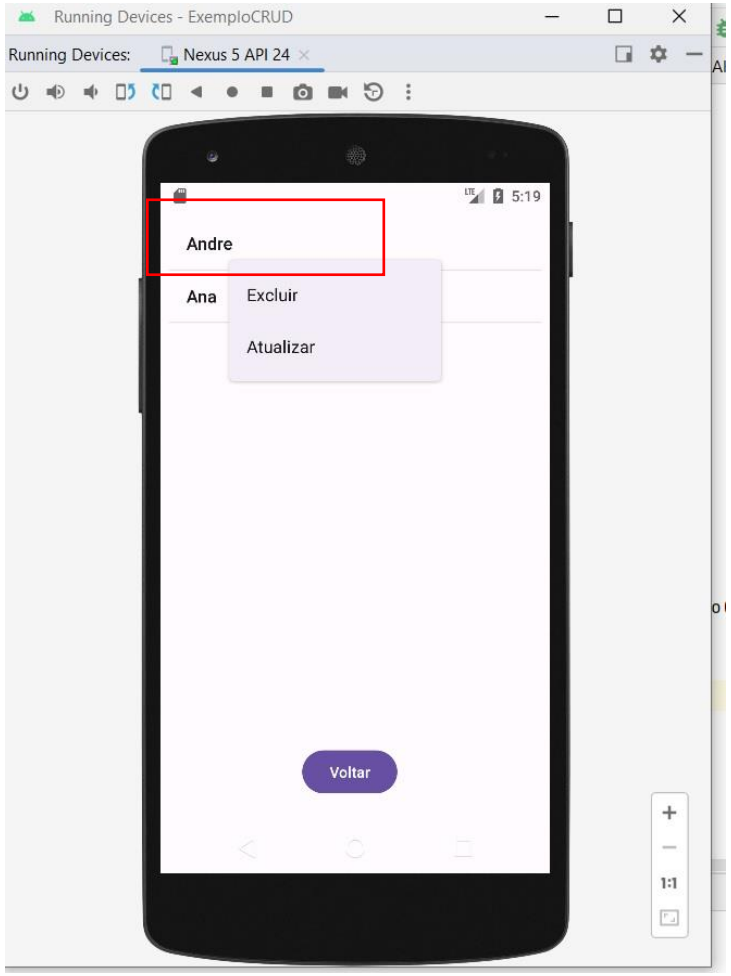
Configurar o evento ‘OnClick’ do botão de menu ‘Excluir’. Verificar se foi incluído também no código do Menu_Contexto.xml

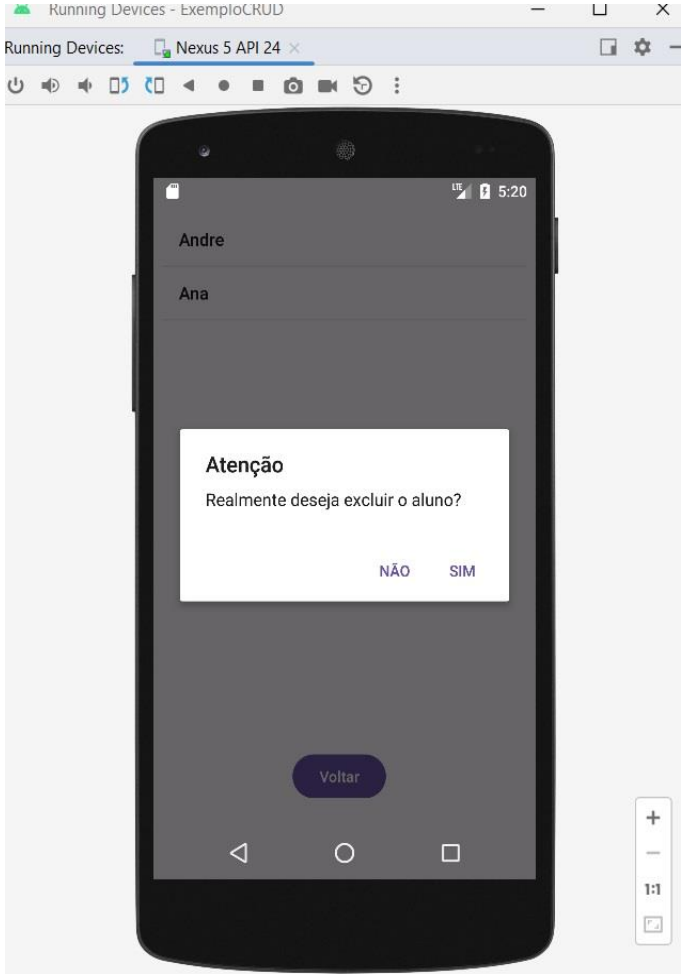
```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:onClick="excluir"
        android:title="Excluir" />
    <item android:title="Atualizar" />
</menu>
```

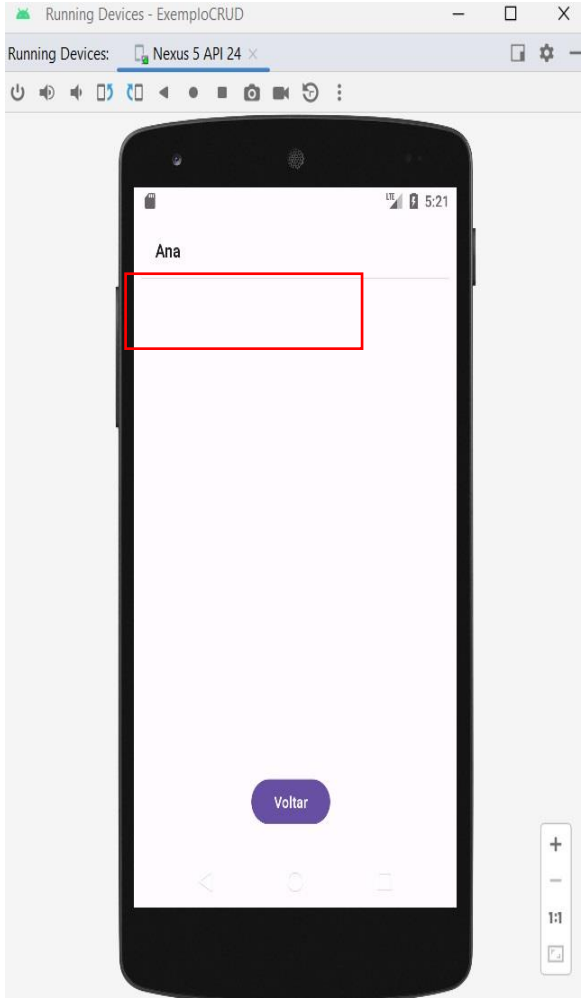


Excluir

Resultado







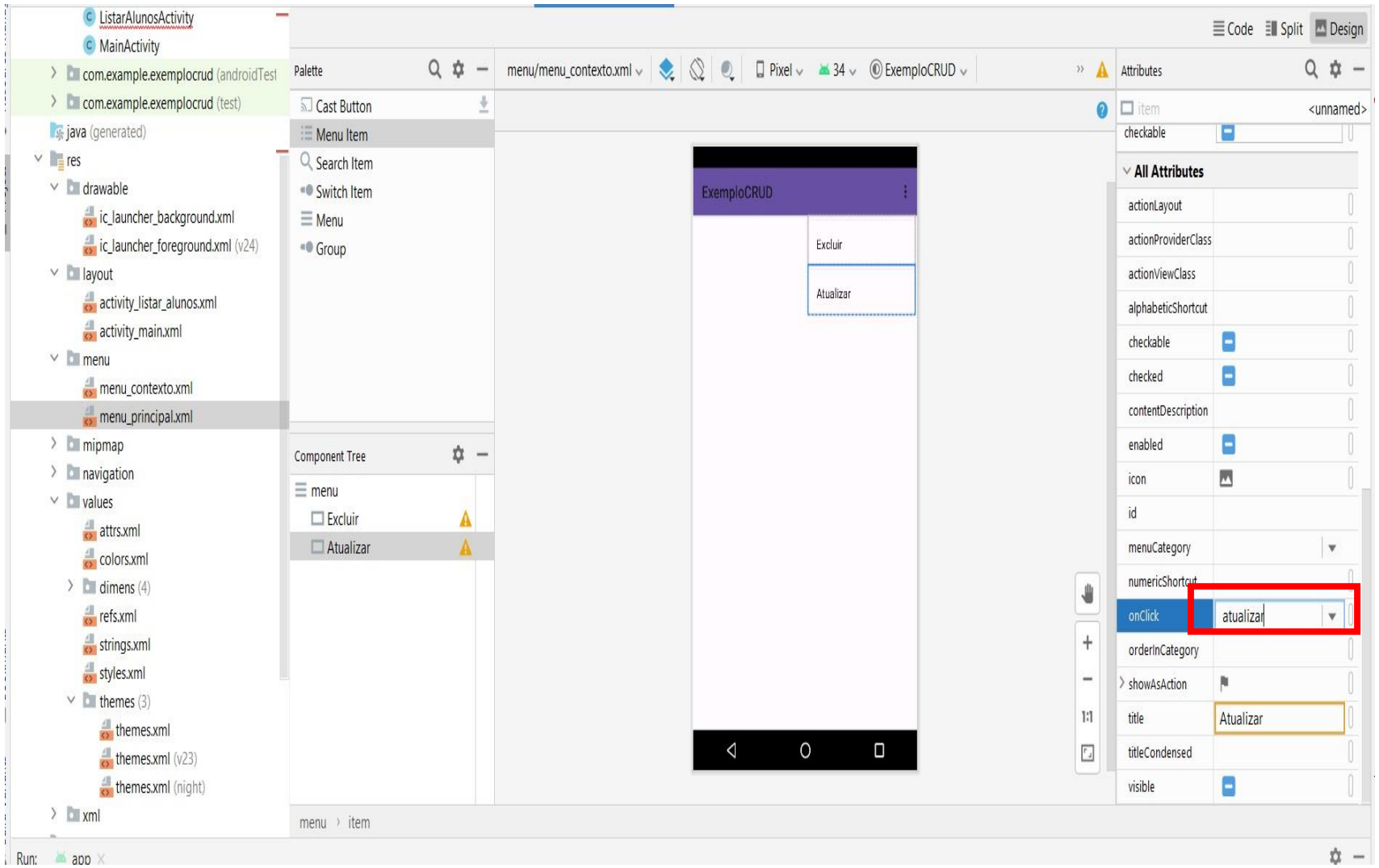
Atualizar

Agora vamos fazer o ‘Atualizar’ para quando o item listado for selecionado, e clicar na opção ‘atualizar’, ele irá levar para tela de ‘cadastro’ chamada neste projeto de ‘MainActivity.java’ com os dados preenchidos para o usuário alterar.

Atualizar

Passo 01: Já vamos deixar pronto o evento para o botão ‘atualizar’ do MenuItem.

Configurar o evento ‘OnClick’ do botão de menu ‘Atualizar’. Verificar se foi incluído também no código do Menu_Contexto.xml

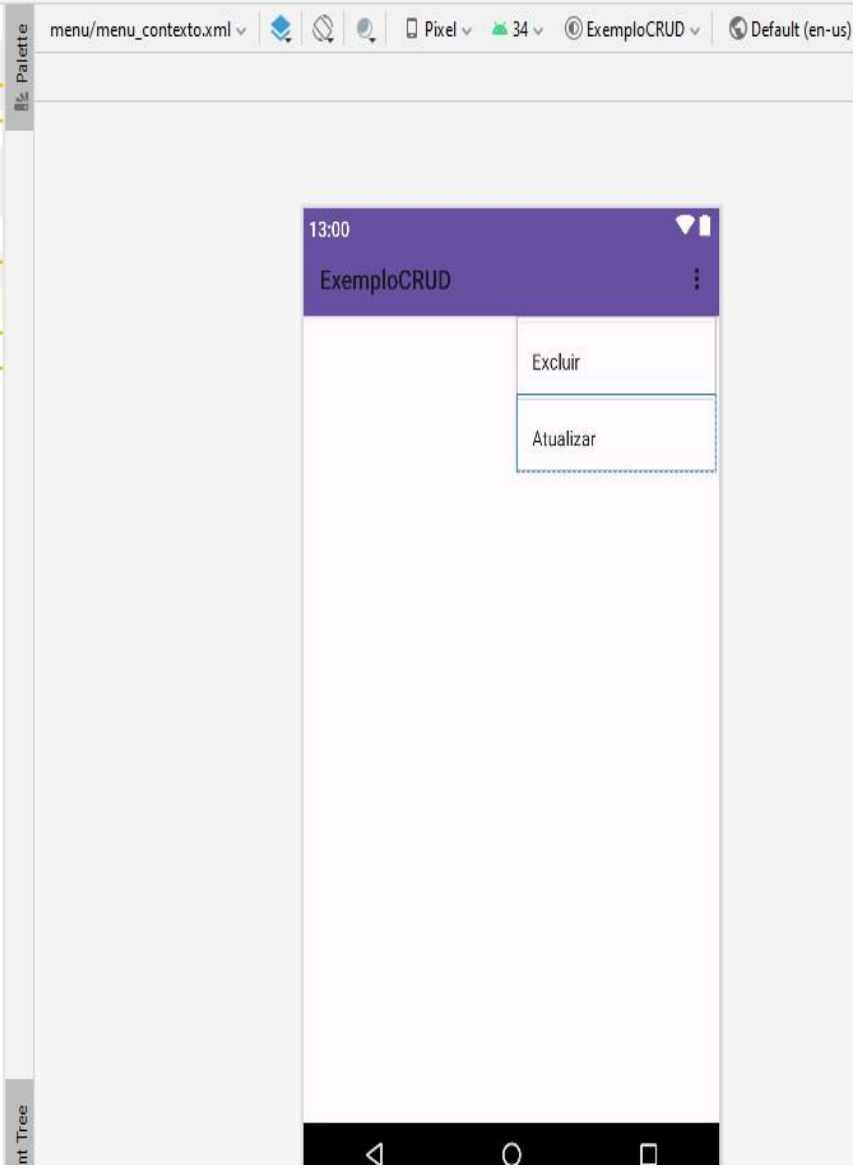


Atualizar

Passo 01: Já vamos deixar pronto o evento para o botão ‘atualizar’ do MenuItem.

Configurar o evento ‘OnClick’ do botão de menu ‘Atualizar’. Verificar se foi incluído também no código do Menu_Contexto.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:onClick="excluir"
        android:title="Excluir" />
    <item
        android:onClick="atualizar"
        android:title="Atualizar" />
</menu>
```



Atualizar

Pronto, agora vamos criar o método ‘Atualizar’ dentro da classe ‘ListarAlunoActivity.java’.
- Segue a mesma lógica do excluir, pois o botão de menu é o mesmo.

```
public void atualizar(MenuItem item) {
    // Obtém informações do item selecionado no menu de contexto.
    // O objeto `menuInfo` contém a posição do item na lista.
    AdapterView.AdapterContextMenuInfo menuInfo = (AdapterView.AdapterContextMenuInfo) item.getMenuInfo();

    // Obtém o aluno que será atualizado a partir da lista filtrada.
    final Aluno alunoAtualizar = alunosFiltrados.get(menuInfo.position);

    // Cria uma Intent para abrir a tela de cadastro (MainActivity).
    // Isso permite reutilizar a mesma tela para edição.
    Intent it = new Intent(this, MainActivity.class);

    // Adiciona o objeto `alunoAtualizar` à Intent, para que os dados sejam
    // carregados na tela de cadastro e possam ser editados.
    it.putExtra("aluno", alunoAtualizar);

    // Inicia a Activity de cadastro (MainActivity) com os dados do aluno selecionado.
    startActivity(it);
}
```

Atualizar

Agora vamos colocar os dados no ‘MainActivity.java’ para receber os dados que vieram do atualizar.

```
//1)
//Declarar antes do método OnCreate(), se ainda não estava declarado
private Aluno aluno = null;
```

```
//2
//Ajustar dentro do OnCreate() para pegar os dados
//PEGAR OS DADOS QUE VEM NO INTENT DO ATUALIZAR
    Intent it = getIntent(); //pega intenção
    if(it.hasExtra("aluno")){
      aluno = (Aluno) it.getSerializableExtra("aluno");
      nome.setText(aluno.getNome().toString());
      cpf.setText(aluno.getCpf());
      telefone.setText(aluno.getTelefone());
    }
```

Atualizar

Agora vamos organizar os dados no ‘MainActivity,java’ para receber os dados que vieram do atualizar.

MainActivity.java | menu_contexto.xml | AlunoDao.java | ListarAlunosActivity.java | activity_listar_alunos.xml | activity_main.xml | Aluno.java | Conex

```java
    private EditText telefone;

    5 usages
    private AlunoDao dao;

    4 usages
    private Aluno aluno = null;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // Chama o método onCreate() da classe pai (AppCompatActivity),
        // que configura aspectos essenciais da Activity, como a criação da janela,
        // a restauração do estado salvo e outras inicializações necessárias do ciclo de vida.

        setContentView(R.layout.activity_main);
        // Define qual arquivo de layout XML será usado para esta Activity,
        // ou seja, infla o layout 'activity_listar_alunos.xml' e o torna a interface exibida na tela.

        //Vinculando os campos do layout com as variáveis do Java
        nome = findViewById(R.id.editNome);
        cpf = findViewById(R.id.editCPF);
        telefone = findViewById(R.id.editTelefone);

        dao = new AlunoDao( context: this);

        //---------------- Código que recebe os dados do Atualizar (ListarAluosActivity)--------------------------------
        Intent it = getIntent(); //pega intenção
        if(it.hasExtra( name: "aluno")){
            aluno = (Aluno) it.getSerializableExtra( name: "aluno");
            nome.setText(aluno.getNome().toString());
            cpf.setText(aluno.getCpf());
            telefone.setText(aluno.getTelefone());
        }


    }
```

Atualizar

Modificar o método ‘salvar’ dentro do ‘MainActivity.java’ para verificar se o aluno (variável criada na classe ‘MainActivity’ e usada para receber dados por intenção do ‘atualizar’, ou é igual a **‘null’, então faz o** cadastro normal.

Se está recebendo algum dado por intenção fazemos a atualização.

```
//método para botão salvar qdo clicado
  public void salvar(View view){
    if(aluno==null) { //SE NAO RECEBEU DADOS DO ATUALIZAR É ESTA COMO 'NULL', CADASTRA O ALUNO
      aluno = new Aluno();
      aluno.setNome(nome.getText().toString());
      aluno.setCpf(cpf.getText().toString());
      aluno.setTelefone(telefone.getText().toString());
      long id = dao.inserir(aluno); //inserir o aluno
      Toast.makeText(this,"Aluno Inserido com sucesso!! com id: ", Toast.LENGTH_SHORT).show();
    }
    else {
      //RECEBENDO DADOS DO ATUALIZAR NA VARIAVEL 'aluno'
      //seta os valores de novo e atualiza
      aluno.setNome(nome.getText().toString());
      aluno.setCpf(cpf.getText().toString());
      aluno.setTelefone(telefone.getText().toString());
      dao.atualizar(aluno); //inserir o aluno
      Toast.makeText(this,"Aluno Atualizado!! com id: ", Toast.LENGTH_SHORT).show();
    }

  }
```

Atualizar

Como na aula passada já havíamos realizado as validações dos dados no método ‘salvar’, dentro da ‘MainActivity’, agora vamos realizar as adaptações sugeridas no slide anterior!! No próximo slide.

Atualizar

Parte I
Método
salvar()
MainActivity
validações

```
//---------------------------//método para botão salvar qdo clicado-----------------------------------------------------------//
public void salvar(View view){

    String nomeDigitado = nome.getText().toString().trim();
    String cpfDigitado = cpf.getText().toString().trim();
    String telefoneDigitado = telefone.getText().toString().trim();

    // Verifica se os campos estão vazios
    if (nomeDigitado.isEmpty() || cpfDigitado.isEmpty() || telefoneDigitado.isEmpty()) {
        Toast.makeText(this, "Preencha todos os campos!", Toast.LENGTH_SHORT).show();
        return;
    }

    // Validação do CPF (verifica se o formato e os dígitos são válidos)
    System.out.println("CPF antes da validação: " + cpfDigitado);
    if (!dao.validaCpf(cpfDigitado)) {
        Toast.makeText(this, "CPF inválido. Digite novamente.", Toast.LENGTH_SHORT).show();
        return;
    }

    // Se for cadastrar novo aluno ou Se for atualizar os dados ignora o CPF se for igual do próprio aluno
    //Se o aluno atualizar um cpf diferente dai sim será verificado
    if(aluno == null || !cpfDigitado.equals(aluno.getCpf())){
        //  verifica se o CPF já existe no banco
        if (dao.cpfExistente(cpfDigitado)) {
            Toast.makeText(this, "CPF duplicado. Insira um CPF diferente.", Toast.LENGTH_SHORT).show();
            return;
        }
    }

    // Validação do Telefone
    if (!dao.validaTelefone(telefoneDigitado)) {
        Toast.makeText(this, "Telefone inválido! Use o formato correto: (XX) 9XXXX-XXXX", Toast.LENGTH_SHORT).show();
        return;
    }
```

Atualizar

Parte II
Método
salvar()
MainActivity

```
// aluno ==null cadastrar, aluno!=null esta recebendo do ListarAlunos
if (aluno == null) {
    // Criar objeto Aluno
    Aluno aluno = new Aluno();
    aluno.setNome(nomeDigitado);
    aluno.setCpf(cpfDigitado);
    aluno.setTelefone(telefoneDigitado);

    // Inserir aluno no banco de dados
    long id = dao.inserir(aluno);

    if (id != -1) {
        Toast.makeText(this, "Aluno inserido com id: " + id, Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Erro ao inserir aluno. Tente novamente.", Toast.LENGTH_SHORT).show();
    }
}
else{
    // Atualização de um aluno existente
    aluno.setNome(nomeDigitado);
    aluno.setCpf(cpfDigitado);
    aluno.setTelefone(telefoneDigitado);

    dao.atualizar(aluno);
    Toast.makeText(this, "Aluno atualizado com sucesso!", Toast.LENGTH_SHORT).show();
}

// Fecha a tela de cadastro e volta para a listagem
finish();
}
```

Atualizar

O próximo passo é criar o método ‘atualizar’ no ‘AlunoDao.java’.

```
//------------------- ATUALIZAR -------------------------------------------------------//
public void atualizar(Aluno aluno){
    ContentValues values = new ContentValues(); //valores que irei inserir
    values.put("nome", aluno.getNome());
    values.put("cpf", aluno.getCpf());
    values.put("telefone", aluno.getTelefone());
    banco.update("aluno", values, "id = ?", new String[]{aluno.getId().toString()});
}
```

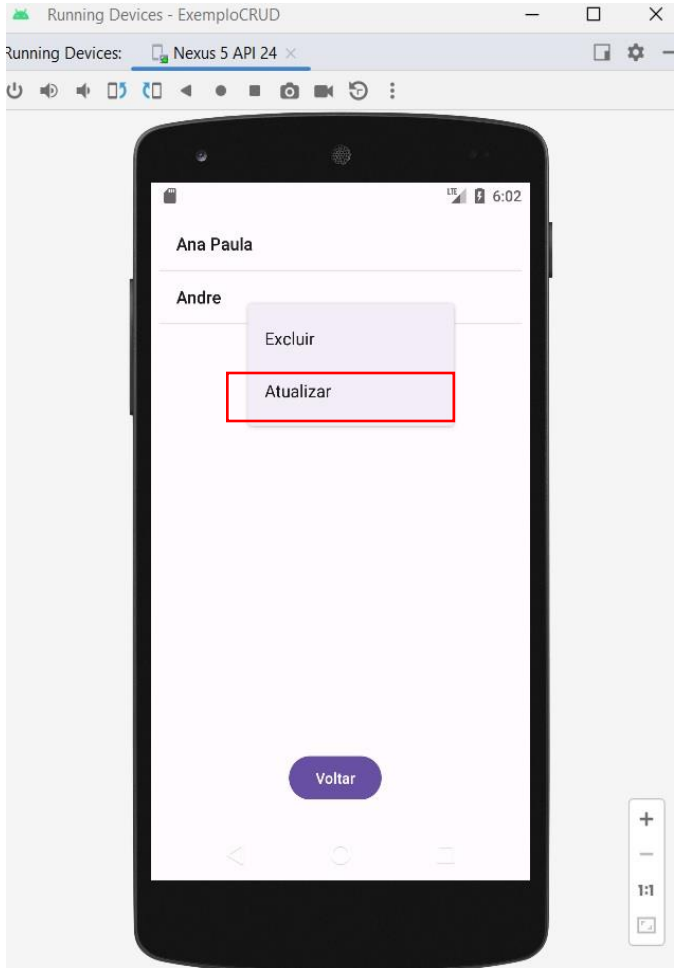
Atualizar

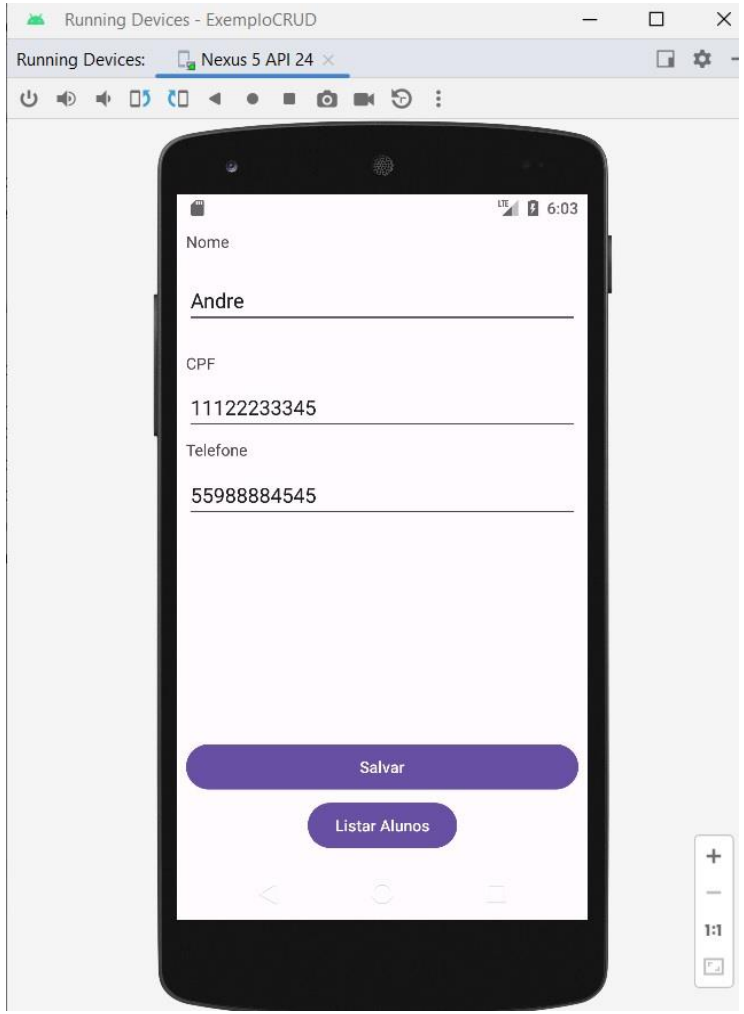
O último passo é usar o método onResume() para atualizar a listview com os alunos atualizados, dentro 'ListarAlunosActivity.java'. Caso contrário a lista vai aparecer desatualizada ao voltar para o listar alunos.

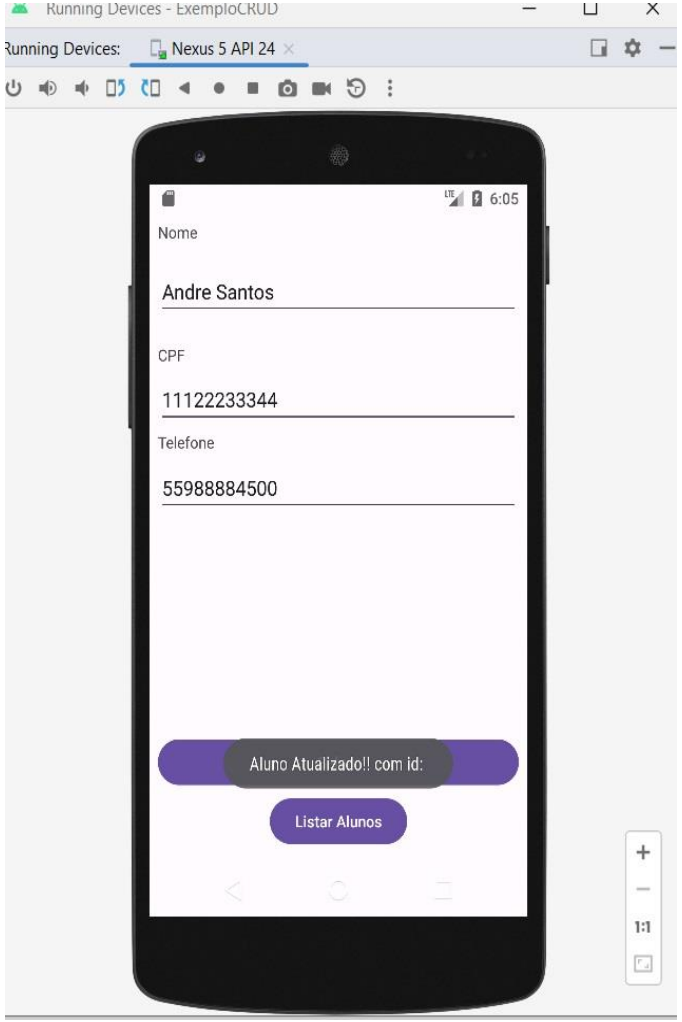
```
//Recarregar a lista de alunos a ser exibida após as modificações do método 'atualizar'
@Override
protected void onResume() {
    super.onResume();
    // Recarrega todos os alunos do banco de dados
    alunos = dao.obterTodos();

    // Limpa a lista filtrada e adiciona os novos alunos
    alunosFiltrados.clear();
    alunosFiltrados.addAll(alunos);

    // Atualiza o adapter da ListView para refletir os novos dados
    ArrayAdapter<Aluno> adaptador = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, alunosFiltrados);
    listView.setAdapter(adaptador);
}
```

Atualizar

Resultado







Dicas

Para visualizar os métodos da classe pai no Android Studio (versão 2022.3.1 ou qualquer outra), existem algumas técnicas que você pode usar:

1. Ver o Código-Fonte da Classe Pai:
Passo 1: Coloque o cursor sobre o nome da classe pai (por exemplo, Activity).
Passo 2: Pressione Ctrl + B (no Windows/Linux) ou Cmd + B (no macOS). Isso abrirá o código-fonte da classe pai, onde você pode ver todos os métodos, variáveis e documentação.

2. Documentação Rápida:
Passo 1: Coloque o cursor sobre o nome de um método ou classe.
Passo 2: Pressione Ctrl + Q (no Windows/Linux) ou Ctrl + J (no macOS). Isso abrirá a documentação rápida que mostra informações sobre a classe, incluindo métodos herdados.

3. Hierarquia de Classes:
Passo 1: Com o cursor sobre o nome da classe, pressione Ctrl + H (no Windows/Linux) ou Cmd + H (no macOS).
Passo 2: Isso abrirá a visão de hierarquia de classes, onde você pode navegar pelas superclasses e ver os métodos herdados.

Dicas

Para visualizar os métodos da classe pai no Android Studio (versão 2022.3.1 ou qualquer outra), existem algumas técnicas que você pode usar:

4. Ver a Implementação do Método Herdado:
Passo 1: Coloque o cursor sobre o método (por exemplo, onCreateContextMenu).
Passo 2: Pressione Ctrl + U (no Windows/Linux) ou Cmd + U (no macOS). Isso abrirá a implementação original do método na classe pai.

Dicas

Como abrir o banco de dados que esta no celular pelo Android Studio:

1) Abra o "Device File Explorer" no Android Studio:
Vá em "View" > "Tool Windows" > "Device File Explorer".
Se não encontrar, tente "Window" > "Device Explorer".

2) Navegue até a pasta do seu app:
Dentro do Device File Explorer, procure:
/data/data/com.example.exemplocrud/ (nome que colocamos ao pacote do projeto)
OBS: Se não aparecer nada, verifique se o app já foi executado pelo menos uma vez no emulador.

3) Dentro dessa pasta, procure databases/
Abra a pasta databases/
Você deve encontrar o arquivo do banco, algo como:
banco.db **('banco' é o nome que demos ao banco de dados, baixar este)**
banco.db-journal

Como abrir o banco de dados que esta no celular pelo Android Studio:

4) Baixar o arquivo do banco para o PC:
- Clique com o botão direito no arquivo e escolha "Save As" para salvar no seu computador.
- Agora, abra o banco com o DB Browser for SQLite ou SQLiteStudio.

Vamos usar o SQLiteStudio, Link para download: https://sqlitestudio.pl/ (instale a ferramenta no seu pc)

Atenção:
Se a pasta databases/ não aparecer
Se a pasta databases/ não estiver visível, pode ser que:

O banco de dados ainda não tenha sido criado.

Execute uma operação no app que force a criação do banco (ex: cadastrar um aluno).
Depois, volte para o Device File Explorer e tente novamente.

Você não tem permissões para acessar /data/data/.
- No emulador, geralmente funciona sem problemas.
- Se estiver usando um celular físico, precisa de root para acessar essa pasta.

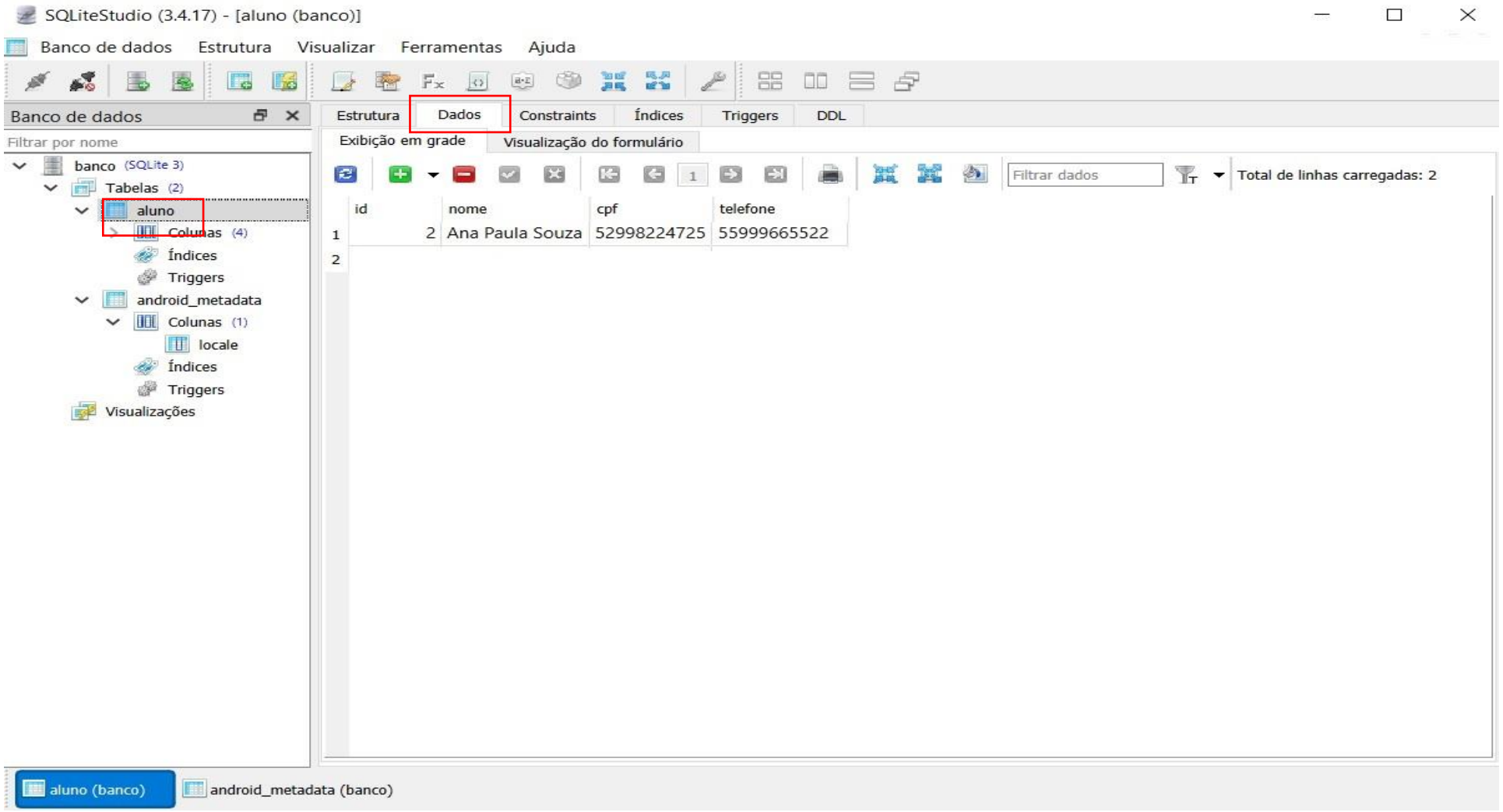
Como abrir o banco de dados que esta no celular pelo Android Studio:

Abrindo o arquivo no SQLite:

Abra o banco.db
- No SQLiteStudio, clique em "Database" > "Add a database".
- Selecione o arquivo banco.db. (ou o nome que você escolheu para seu banco no projeto)

Como abrir o banco de dados que esta no celular pelo Android Studio:

Abrindo o arquivo no SQLite: Selecionar a tabela e clicar em dados para visualizar.



Atualizar/Excluir

Fim

Exercícios:

Descrição

Referências:

LECHETA, Ricardo R. Google Android: aprenda a criar aplicações para dispositivos móveis com o android SDK. São
Paulo: Novatec, 2009.
TALUKDER, Asoke; YAVAGAL, Roopa. Mobile computing. New Delhi - India: McGraw-Hill, 2007.
SILVA, D. Desenvolvimento para dispositivos móveis. Pearson, 2017.
ANDROID. Android Developers. Disponível em https://developer.android.com. Acesso em agosto de 20018. 2018.
MIKKONEN, T. Programming mobile devices: an introduction for practitioners. Chichester,
England: Wiley, 2007.
ROGERS, Rick et al. Desenvolvimento de aplicações Android. O'Reilly: Novatec, 2009.
SILVA, D. Arquitetura para computação móvel. Pearson, 2018.
YAVAGAL, Roopa R.; TALUKDER, Asoke K. Mobile computing: technology, applications, and service creation. New
York, NY: McGraw-Hill, 2007.

Obrigado pela sua atenção!!

Contato:
Email: **andre.flores@ufn.edu.br**